



Consultas Express

**Asistente de clasificación automática
para respuestas rápidas**

Informe ejecutivo del proyecto final — Curso IA práctica para crear
soluciones de negocio con Google Antigravity

Autor: Marco Antonio Chamorro Villoria

Curso: IA Práctica · Antigravity

Fecha: Junio 2026

Versión: MVP v1.0

1. Resumen Ejecutivo

Las pequeñas y medianas empresas dedican una media de **2 a 3 horas diarias** a responder consultas repetitivas de clientes a través de WhatsApp, email o formularios web. Preguntas sobre precios, horarios, estado de envíos o disponibilidad de productos se responden una y otra vez con la misma información, consumiendo tiempo valioso y generando fatiga en los empleados.

Consultas Express es una aplicación web ligera — sin servidor, sin instalación y sin dependencias externas — que permite al personal de una PYME pegar una consulta de cliente y obtener, en segundos, una **clasificación automática por palabras clave** junto con una **respuesta base editable**. La herramienta no sustituye al empleado: acelera su trabajo.



El prototipo se ha desarrollado siguiendo la metodología de 8 fases del curso, implementado con tecnologías web estándar (HTML, CSS, JavaScript vanilla), desplegado en GitHub Pages y diseñado bajo una estética retro-futurista synthwave. **Coste total de infraestructura: 0€.**

2. Descripción del Problema

2.1 El problema

Una PYME tipo (asesoría, clínica, taller, tienda online, imprenta) recibe una media de **20 a 40 consultas diarias** a través de múltiples canales digitales. La mayoría son repetitivas y caen en pocas categorías:

Categoría	Frecuencia	Ejemplo
Precios y presupuestos	25%	"¿Cuánto cuesta el pack básico?"
Horarios y disponibilidad	20%	"¿A qué hora abris los sábados?"
Estado de envíos	18%	"¿Cuánto tarda un pedido a Vizcaya?"
Devoluciones y cambios	15%	"Quiero devolver un producto"
Información de producto	15%	"¿Tenéis el modelo P320 en negro?"
Facturación y pagos	7%	"Necesito la factura del pedido"

Datos estimados a partir del seed data de 30 consultas ficticias y observación de negocio típico PYME.

2.2 Stakeholders

Stakeholder	Necesidad	Problema actual	Impacto
Empleado atención cliente	Responder rápido y bien	Fatiga por repetición, tiempo perdido	Desmotivación, errores
Cliente	Respuesta clara y rápida	Espera excesiva, respuestas genéricas	Insatisfacción, fuga

Gerente / PYME	Eficiencia operativa	Coste elevado, equipo distraído	Menos productividad
----------------	----------------------	---------------------------------	---------------------

2.3 Coste actual estimado

Concepto	Valor	Fuente
Consultas diarias	30	Estimación
Tiempo por consulta	2 min	Estimación
Tiempo total diario	60 min	Cálculo
Coste/hora empleado	18€	Estimación (perfil atención cliente)
Coste semanal	90€	Cálculo (5h × 18€)
Coste anual	4.320€	Cálculo (240h × 18€)

Consultas Express · Informe Ejecutivo · 2

2.4 Nuevos riesgos que podría generar la solución

Posible problema	Causa	Medida preventiva
Dependencia de la herramienta	Empleados confían ciegamente en la app	Formación: "la app ayuda, no decide". Indicador de confianza visible
Resistencia al cambio	Equipo acostumbrado a proceso manual	Implementación gradual, empezar con 1 empleado
Falsa sensación de precisión	Confianza del 90% no implica corrección	Nunca enviar automáticamente. Revisión humana obligatoria
Obsolescencia de respuestas base	Precios, horarios cambian	Revisión mensual de matriz de respuestas (texto plano, fácil editar)

3. Caso de Uso y Criterios de Éxito

3.1 Caso de uso principal

Usuario: Empleado de atención al cliente de una PYME.

Necesidad: Responder rápidamente a consultas repetitivas sin redactar cada respuesta desde cero.

Qué introduce: El texto literal de la consulta del cliente (copiado de WhatsApp, email o formulario web).

Qué obtiene: Categoría detectada + nivel de confianza (0-99%) + respuesta base editable en un textarea.

3.2 Ejemplo de uso

Input: "Buenas, ¿cuánto cuesta el pack básico mensual?"

Proceso: El sistema normaliza el texto (minúsculas, sin tildes), busca coincidencias en la matriz de palabras clave. Encuentra "cuanto cuesta" (precio, peso 2). Score total: precio=2. Sin otras coincidencias significativas.

Output: Categoría: **PRECIO** · Confianza: 95%

Respuesta sugerida: "Gracias por tu consulta. El precio actual es [X]€ IVA incluido. ¿Necesitas más información?"

Acción del empleado: Edita "[X]" por "29,99", revisa el texto, click "Usar respuesta", envía al cliente.

Fuente: dato simulado a partir del seed data (data.json). En producción se sustituirá por datos reales.

3.3 KPIs / Criterios de éxito

Indicador	Actual	Objetivo	Medición
Tiempo medio por consulta	2 min	30 seg	Tiempo desde pegar consulta hasta respuesta lista

Precisión de clasificación	—	>75%	% consultas con categoría correcta
Respuestas usadas sin editar	—	>40%	% respuestas aceptadas sin modificación
Tiempo ahorrado semanal	0 min	>90 min	(consultas×2min) – (consultas×30seg)
Usabilidad sin formación	—	Sí	Empleado no técnico usa sin ayuda
Consultas gestionadas/día	30	30+	Volumen máximo. Más rápidas, misma capacidad

4. Solución Propuesta

4.1 Descripción

Aplicación web de una sola página (SPA) que permite al empleado pegar una consulta de cliente y obtener, en menos de un segundo, una clasificación automática por categoría y una respuesta base editable. Diseñada como herramienta de apoyo, no como sustituto de la revisión humana.

4.2 Arquitectura

Usuario → Formulario texto → **Clasificador** (keywords + scoring) → Categoría + respuesta sugerida → Edición humana → Respuesta final

4.3 Tecnologías utilizadas

Tecnología	Rol
HTML5 + CSS3	Estructura y diseño synthwave
JavaScript vanilla (ES6+)	Clasificación, persistencia, interacción
Google Fonts (Orbitron + Plus Jakarta Sans)	Tipografía retro-futurista
localStorage	Persistencia en navegador
GitHub Pages	Despliegue estático gratuito
Cero dependencias externas	Sin React, Node, npm, servidor, BD

4.4 Principios de diseño

- **Sin registro** — uso interno, sin autenticación
- **Sin backend** — toda la lógica en frontend
- **Zero dependencias** — funciona offline tras carga inicial
- **Estética synthwave 80s** — interfaz memorable, no genérica
- **Datos seed precargados** — 30 consultas ficticias para demo inmediata

4.5 Estructura del prototipo

```
/consultas-express
├─ index.html    (app completa: HTML+CSS+JS)
├─ data.json      (seed 30 consultas ficticias)
└─ informe-ejecutivo-consultas-express.html (este documento)
```

4.6 Limitaciones del MVP

- Clasificador por palabras clave (no IA semántica) — puede fallar en consultas ambiguas
- Un solo usuario (localStorage) — no hay sesiones separadas
- Sin integración con canales reales (WhatsApp, email) — el empleado copia y pega manualmente
- Las respuestas base deben actualizarse manualmente si cambian precios o políticas

5. Proceso de Construcción — PoC

5.1 Cómo se construyó

El prototipo se desarrolló en Google Antigravity mediante un proceso iterativo de 4 ciclos:

Ciclo 1 — Estructura base: Prompt solicitando una aplicación web SPA para clasificar consultas. Antigravity generó la estructura HTML con formulario, botón de envío y zona de resultados. Diseño inicial genérico (tema claro).

Ciclo 2 — Clasificador: Se solicitó explícitamente un sistema sin API externa, usando matriz de palabras clave con scoring. Antigravity generó el algoritmo de normalización, la matriz de 7 categorías y el cálculo de confianza. Se probó con 5 ejemplos y se ajustaron pesos para evitar falsos positivos.

Ciclo 3 — Diseño synthwave: Rediseño completo de la interfaz: fondo oscuro, neón rosa/cian, Orbitron, glassmorphism, rejilla retrowave. Antigravity generó el CSS con variables, animaciones y efectos glow.

Ciclo 4 — Persistencia y pulido: localStorage, data.json, export JSON, vista estadísticas, modal detalle, filtros de historial, búsqueda en vivo.

Consultas Express · Informe Ejecutivo · 4

5.2 Prompts utilizados

Prompts representativos reconstruidos a partir del proceso. Al ser desarrollo asistido por Antigravity, cada prompt fue iterado y refinado.

Prompt 1 — Base:

"Crea una aplicación web en un solo archivo HTML que permita al usuario pegar una consulta de cliente, la clasifique automáticamente en categorías (precio, horario, envío, devolución, producto, factura, otro) usando palabras clave y genere una respuesta base editable."

Prompt 2 — Algoritmo:

"Implementa el clasificador con JavaScript vanilla. Usa una matriz de palabras clave con pesos. La confianza debe ser el porcentaje que representa la categoría ganadora sobre el total de puntos. Incluye normalización de texto (minúsculas, sin tildes)."

Prompt 3 — Diseño:

"Rediseña la interfaz con estética synthwave 80s retro-futurista: fondo púrpura oscuro #0a0015, neón rosa #ff2a75, neón cian #00e5ff. Orbitron para títulos, Plus Jakarta Sans para cuerpo. Efecto glassmorphism, rejilla retrowave de fondo."

Prompt 4 — Persistencia:

"Añade persistencia con localStorage. Carga 30 consultas de ejemplo desde data.json al iniciar. Añade historial con búsqueda, filtros por estado y exportación JSON."

5.3 Inspiración y referencias

El diseño y la funcionalidad se inspiraron en:

- **OutRun / Hotline Miami** — estética neón sobre fondos oscuros, rejillas de perspectiva
- **Nintendo.com (2001)** — análisis de diseño del curso (beveled plates, paleta periwinkle/carbon), reinterpretado en clave moderna
- **Glassmorphism** — tendencia UI 2022-2024 (paneles semitransparentes con blur)
- Proyectos de referencia: *support-tickets* (CRUD + estados), *movies-dataset* (filtros + gráficos), *st-Lezo* (tabs + cards)

5.4 Decisiones de diseño y dificultades

Decisión	Alternativa descartada	Motivo
Clasificador por keywords	Gemini API, Hugging Face	MVP sin backend ni API keys. Keyword-matching suficiente para PYME pequeña
localStorage	Supabase, Firebase	Zero infraestructura. El empleado usa un solo equipo
HTML único	React, Vue, Streamlit	Despliegue inmediato sin build. Cero dependencias
Diseño synthwave	Material Design, Bootstrap	Diferenciación. App uso interno, estilo memorable ayuda adopción

Dificultad principal: diferenciar categorías cercanas ("precio del envío" → ¿precio o envío?).

Solución: mayor peso a palabras específicas. Si ambas aparecen, gana la categoría con más coincidencias totales.

6. MVP y Prototipo Desarrollado

6.1 Vistas funcionales

Vista	Funcionalidad
Nueva consulta	Formulario texto + clasificación automática + respuesta editable + acciones (usar, reclasificar, descartar)
Historial	Listado completo con búsqueda en vivo, filtros por estado y exportación JSON
Estadísticas	Métricas clave + distribución por categoría + evolución diaria 7 días

6.2 Flujo de usuario

- Empleado abre la aplicación
- Pega la consulta del cliente en el textarea
- Click **Clasificar** — la app muestra categoría + confianza + respuesta base
- Empleado edita la respuesta si es necesario
- Selecciona **Usar respuesta** (marca como respondida)
- La consulta se guarda automáticamente en el historial

6.3 Pantallas clave

Elemento	Descripción
Hero + Form	Texto neón con grid synthwave, textarea con foco cian, botón submit rosa con glow
Result card	Badge categoría (color por tipo), confianza, textarea editable, botones acción
Cards historial	Borde izquierdo coloreado por categoría, badge, texto, fecha, estado, flecha detalle
Modal detalle	Información completa, textarea editable, botones (responder, guardar, eliminar)
Dashboard stats	4 tarjetas métricas, barras categoría, gráfico evolución diaria

6.4 Diseño visual

Estética **synthwave 80s**: fondo púrpura oscuro (#0a0015), neón rosa (#ff2a75) y cian (#00e5ff), Orbitron para títulos, Plus Jakarta Sans para cuerpo, glassmorphism (blur + transparencia), botones con gradiente rosa, sombras de neón, rejilla retrowave de fondo. Sin imágenes externas, sin iconos CDN — solo emojis y CSS puro.

6.5 Persistencia y datos

Componente	Rol
data.json	Seed 30 consultas ficticias (fetch al iniciar si localStorage vacío)
localStorage	Persistencia de todas las consultas (seed + nuevas)
Export JSON	Backup completo para restauración futura

7. Evaluación y Resultados

7.1 Casos de prueba

Se probaron 7 consultas representativas (una por categoría + un caso ambiguo):

#	Input	Esperado	Obtenido	Conf.	Val.
1	"¿Cuánto cuesta el pack básico?"	precio	precio	95%	✓
2	"¿A qué hora abris los sábados?"	horario	horario	97%	✓
3	"¿Cuánto tarda un envío a Vizcaya?"	envio	envio	94%	✓
4	"Quiero devolver un producto"	devolucion	devolucion	93%	✓
5	"¿Tenéis el modelo P320 en negro?"	producto	producto	96%	✓
6	"Quiero trabajar con vosotros"	otro	otro	76%	✓
7	"¿Cuánto vale el envío urgente?"	precio/envio	envio	62%	⚠

Fuente: pruebas manuales durante el desarrollo. Inputs representativos de consultas reales PYME. Caso 7: confusión esperable, marcado como "revisar" por baja confianza.

7.2 Resultados

Métrica	Antes (manual)	Después (con app)	Mejora
Tiempo por consulta	~120 seg	~30 seg	-75%
Tiempo 30 consultas	60 min	15 min	-45 min/día
Precisión (7 casos)	—	86% (6/7)	—
Errores en respuesta	Posibles (fatiga)	Reducidos (base + revisión)	Cualitativo

7.3 Evaluación del KPI principal

El KPI principal era reducir el tiempo de respuesta de 2 minutos a 30 segundos (-75%). El prototipo lo consigue en condiciones controladas. La precisión del clasificador (86%) supera el objetivo del 75%. El caso 7 (ambigüedad precio/envío) muestra una limitación del enfoque por keywords que se abordará ampliando la matriz.

7.4 Ajustes realizados

Problema	Causa	Ajuste
Consultas con tildes no clasificaban	Comparación literal	Añadir normalize('NFD') + replace diacríticos
Envío permitía consultas vacías	Falta validación	Añadir trim() + toast si vacío
Confusión precio/envío	Pesos iguales	Categoría con más coincidencias gana. Si empate, primera detectada

Placeholder [X] sin reemplazar	No se extraía número	Añadir regex para detectar cantidades
Seed data se perdía al recargar	Cargado desde variable JS	Mover a data.json con fetch + localStorage

8. Riesgos y Uso Responsable

8.1 Riesgos

Riesgo	Prob.	Impacto	Medida de control
Respuesta incorrecta	Media	Alto	Supervisión humana obligatoria. La respuesta nunca se envía automáticamente. El empleado revisa y edita antes de usar
Categorización errónea	Media	Medio	Indicador de confianza visible + botón "Reclasificar"
Fuga datos clientes	Baja	Alto	Sin backend ni servidor. Datos solo en localStorage del navegador. No se transmiten a terceros
Pérdida datos (caché)	Alta	Medio	Export JSON disponible. Dato no crítico (solo histórico)
Fallo técnico (JS)	Baja	Alto	Herramienta de apoyo. Sin ella el proceso manual sigue funcionando
Dependencia navegador	Media	Bajo	Compatible Chrome, Firefox, Edge, Safari actuales (APIs estándar)

8.2 Privacidad, supervisión humana y transparencia

- **Supervisión humana obligatoria:** toda respuesta debe ser revisada antes de enviarse. La app no envía nada automáticamente
- **Transparencia:** indicador de confianza visible (0-99%) en cada clasificación. La interfaz dice "Clasificación automática por palabras clave", no "IA"
- **Edición siempre posible:** la respuesta se muestra en textarea editable, nunca como texto fijo
- **Control de datos:** export JSON disponible, botón reset con confirmación
- **Sin automatización peligrosa:** la app no hace llamadas, no envía mensajes, no modifica datos externos

8.3 Límites de la solución

- No debe usarse para consultas complejas o con datos sensibles (salud, financieros) sin supervisión experta
- No debe enviar respuestas automáticamente (la app no lo permite por diseño)
- No es un chatbot — requiere que el empleado copie y pegue la respuesta manualmente
- Las respuestas base pueden quedar desactualizadas si no se revisan periódicamente

8.4 Sesgos potenciales

Sesgo	Descripción	Mitigación
Desbalance de categorías	Categorías con más palabras clave tienen más probabilidad de ser seleccionadas	Revisar y equilibrar la matriz periódicamente
Lenguaje coloquial	Consultas informales o con errores ortográficos reducen precisión	Añadir variantes coloquiales y expandir sinónimos

Falso positivo en "otro"	Consultas ambiguas sin keywords caen en "otro" por defecto	Umbral mínimo de confianza; si no se alcanza, marcar como "revisar"
--------------------------	--	---

9. Plan de Implantación y Hoja de Ruta

9.1 Pasos para llevar la solución a producción

Fase	Acción	Tiempo	Objetivo
1	Validar prototipo con 2-3 usuarios internos	Semana 1	Detectar errores UX y mejorar flujo
2	Expandir matriz de palabras clave con casos reales	Semana 2	Subir precisión de 86% a >90%
3	Añadir categorías nuevas detectadas en validación	Semana 2	Cubrir más tipos de consulta
4	Prueba piloto (1 semana, 1 empleado)	Semana 3	Validar en entorno real
5	Revisar privacidad y permisos	Semana 3	No almacenar datos sensibles
6	Despliegue completo (equipo <5 personas)	Semana 4	Uso diario en producción

9.2 Hoja de ruta

Fase	Acción	Objetivo
Semana 1-2	Validación interna + ajuste matriz	Prototipo estable para piloto
Semana 3	Piloto con 1 empleado + revisión privacidad	Validación real
Semana 4	Despliegue completo + formación equipo	Producción
Mes 2	Evaluar integración Gemini API (IA real)	Mejorar precisión
Mes 3	Valorar deep links WhatsApp/email	Reducir un paso más

9.3 Mejoras futuras

Mejora	Prioridad	Esfuerzo	Impacto
Integración Gemini API (IA real)	Alta	Media	Alto — precisión >90%
Enlaces directos WhatsApp/email	Media	Baja	Alto — reduce un paso
Soporte multiusuario	Media	Alta	Medio — útil equipos >3
Panel tendencias mensuales	Baja	Media	Medio — info negocio

9.4 Coste de implantación

Concepto	Coste
Infraestructura (GitHub Pages)	0€

Dominio personalizado (opcional)	~10€/año
Formación del equipo	30 min presencial
Mantenimiento mensual	1-2 h revisión matriz
Total anual	0-10€

10. Entrega

Componente	Descripción	URL / Archivo
Prototipo funcional	Aplicación web en GitHub Pages	marconato78.github.io/consultas-express
Repositorio	Código fuente completo	github.com/Marconato78/consultas-express
Informe ejecutivo	Este documento HTML imprimible	informe-ejecutivo-consultas-express.html
Informe ejecutivo	Este documento PDF imprimible	Informe Ejecutivo Consultas Express.pdf
Datos de ejemplo	30 consultas ficticias JSON	data.json

11. Conclusión

Consultas Express demuestra que una pequeña herramienta, construida sin presupuesto ni infraestructura, puede generar un ahorro estimado de **~3.240€ anuales** y recuperar **~180 horas de trabajo** para una PYME media. El prototipo es funcional, está desplegado y listo para prueba piloto real.

El proyecto ha seguido la metodología de 8 fases: desde la identificación del problema (fricción en atención al cliente), pasando por el análisis de costes, el diseño del MVP, la construcción iterativa en Google Antigravity, la evaluación con casos de prueba, la identificación de riesgos, y la planificación de la hoja de ruta.

La principal lección aprendida es que **no toda solución necesita IA avanzada** para aportar valor. Un clasificador por palabras clave bien diseñado puede cubrir el 80% de los casos de una PYME pequeña, con la ventaja de ser transparente, predecible y sin coste de API. La integración de IA real (Gemini) queda como mejora natural para la siguiente iteración.

El siguiente paso lógico es validar el prototipo con usuarios reales durante una semana y ajustar la matriz de palabras clave con los datos obtenidos.

Anexo A — Prompts utilizados en Google AntigraVity

Los prompts son representativos del proceso real de desarrollo. Fueron refinados mediante iteraciones sucesivas en AntigraVity.

Prompt de estructura base

```
"Crea una aplicación web en un solo archivo HTML que:  
- Tenga un formulario donde pegar consultas de clientes  
- Clasifique la consulta en categorías: precio, horario, envío, devolución, producto, factura, otro  
- Genere una respuesta base editable  
- Guarde el historial en localStorage  
- Use un diseño synthwave 80s con neón rosa y cian"
```

Prompt de algoritmo de clasificación

```
"Implementa un clasificador por palabras clave en JavaScript. Normaliza el texto (minúsculas, sin tildes).  
  
Usa una matriz de reglas: cada categoría tiene un array de palabras clave con peso 2.  
Calcula la confianza como: score_categoria / score_total * 100.  
Si la confianza es menor a 60%, marca la consulta como 'revisar'."
```

Prompt de persistencia y datos

```
"Separa los datos de ejemplo en un archivo data.json con 30 consultas ficticias.  
  
Carga data.json mediante fetch al iniciar la app. Si localStorage está vacío, guarda los datos seed.  
  
Añade botones de exportación: JSON para backup/restore."
```

Prompt de diseño synthwave

```
"Rediseña la interfaz con estética synthwave 80s. Fondo púrpura oscuro #0a0015, neón rosa #ff2a75, neón cian #00e5ff. Orbitron display, Plus Jakarta Sans body. Glassmorphism en paneles. Botones gradiente rosa. Rejilla retrowave de fondo."
```

Resumen del proyecto

Campo	Valor
Nombre	Consultas Express
Alumno	Marco Antonio Chamorro Villoria

Contexto	PYME — atención al cliente
Problema	Tiempo excesivo respondiendo consultas repetitivas
Solución	App web clasificación automática + respuestas base
Herramientas	HTML, CSS, JS, GitHub Pages, Antigravity
Demo	marconato78.github.io/consultas-express
Repo	github.com/Marconato78/consultas-express

Consultas Express · Informe Ejecutivo · 11